Multimedia Systems

Simplified MPEG-4 Video Encoder Pipeline

Mini Project — Multimedia Systems
Implementation B — Procedural / functional design

Réalisé par : BENLALAM Mohamed Rayane — 222231363816
AKKOUCHE Mehdi — 222231370206
Groupe : M1 IL G3

1. Introduction

Video compression is at the heart of modern multimedia systems. Every video that is streamed, recorded or shared depends on a codec — a system that encodes raw visual data into a compact bitstream and decodes it back into images. This mini-project implements a simplified yet complete MPEG-4-like encoder pipeline in Python, from raw image frames to a compressed binary file that can be decoded back into images.

All codec functionality is organised in a single module (mpeg_codec.py) with top-level functions. The entropy coding stage uses a zigzag scan followed by RLE and Huffman coding, mirroring the standard JPEG/MPEG entropy coding pipeline.

2. Pipeline Description

The encoder consumes a sequence of BGR image frames and emits a single compressed .bin bitstream. Five stages are applied for each input frame:

Pre-processing (Part 1)

Each BGR frame is converted to YCbCr (BT.601) using OpenCV. Chroma planes (Cb, Cr) are downsampled by 2 in both axes using a 2x2 box filter — standard 4:2:0 layout. This exploits the human eye's lower sensitivity to colour compared to brightness, reducing chroma data by 75%.

Intra-frame coding — I-frames (Part 2)

Every G-th frame is coded independently. For each 8x8 block of each YCbCr plane:

1. The block is centred around zero (subtract 128). 2. The 2D DCT-II is applied via cv2.dct — transforming pixel values into frequency coefficients. Low frequencies concentrate in the top-left corner; high frequencies in the bottom-right. 3. Each DCT coefficient is divided by the corresponding entry of the quantisation matrix Q and rounded — this is the lossy step, producing many zeros at high frequencies. 4. The 8x8 coefficient matrix is read in zigzag order, grouping the zeros at the end. 5. RLE encodes runs of consecutive zeros as (zero_count, value) pairs, with a special EOB (End-Of-Block) symbol (0,0) to terminate each block. 6. Huffman coding assigns shorter codes to more frequent RLE symbols. The decoder inverts each step: Huffman decode -> RLE decode -> zigzag unscan -> dequantise -> IDCT -> add 128.

Inter-frame coding — P-frames (Part 3)

Every other frame is predicted from the previously reconstructed frame. The luma plane is divided into 16x16 macroblocks. For each macroblock, the Three-Step Search (TSS) algorithm finds the best matching block in the reference frame within a +-S pixel search window. The residual (current - prediction) is then encoded with the same DCT + quantisation + zigzag + RLE + Huffman pipeline as I-frames. Chroma residuals reuse the luma motion vectors at half resolution.

Entropy coding (Part 4)

The complete bitstream (motion vectors + encoded planes) is assembled and written directly to the .bin output file. A magic header (SM42) and struct-packed metadata allow the decoder to parse the file without any external library.

Evaluation & Visualisation (Part 5)

Compression ratio and per-frame PSNR are computed against the originals. A single matplotlib figure (pipeline.png) visualises all five stages.

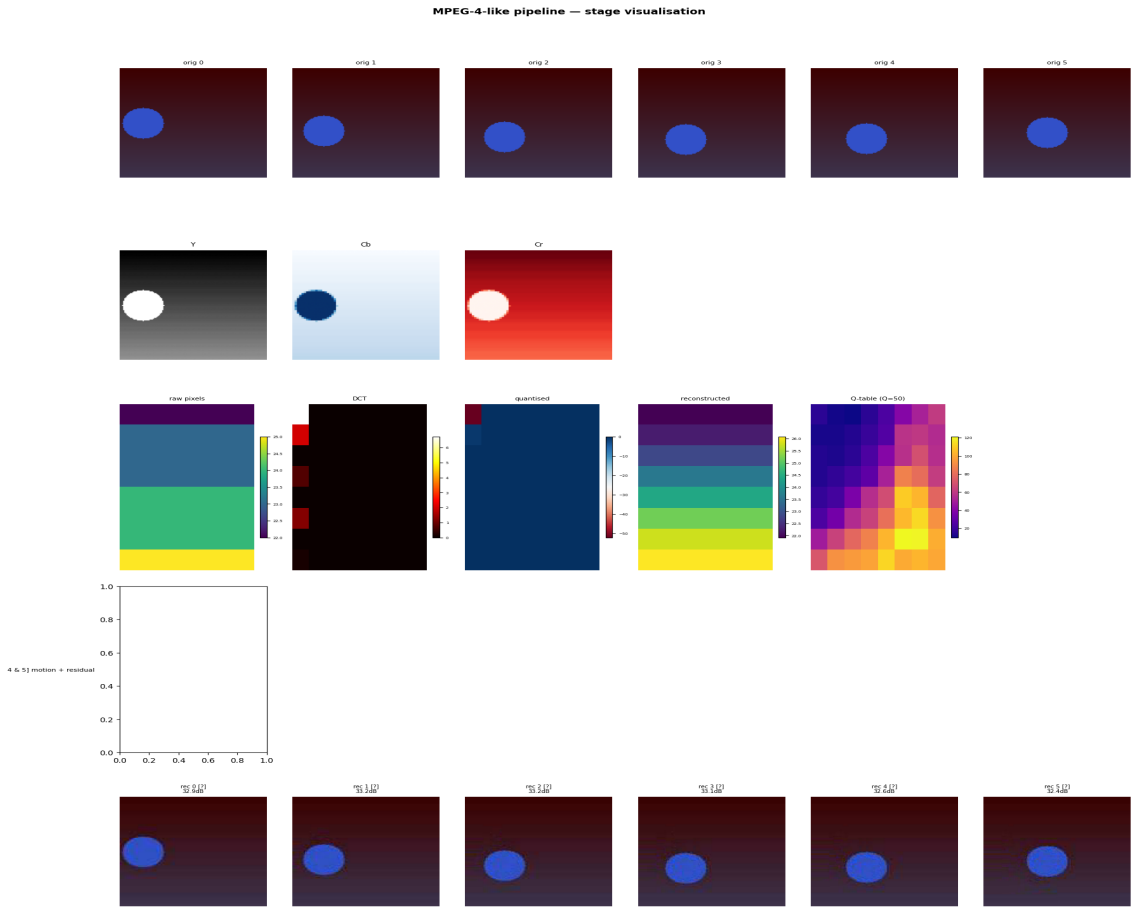

Figure 1 — Pipeline visualisation: original frames (moving circle), YCbCr channels, 8x8 DCT & quantisation, motion vectors on a P-frame, reconstructed frames.

3. Design Choices & Justification

YCbCr (BT.601) + 4:2:0 chroma subsampling. Discards 75% of the chroma samples at no visible cost, exploiting the eye's lower sensitivity to colour detail than brightness.

8x8 block DCT via OpenCV cv2.dct. The canonical JPEG/MPEG block size. OpenCV's DCT-II is BLAS-accelerated and matches standard JPEG/MPEG semantics.

Scaled JPEG quantisation tables. Standard luma/chroma matrices with frequency-dependent weighting. Scaling formula: s = 5000/Q for Q<50, else 200-2Q.

Zigzag scan. Reading the 8x8 quantised block in zigzag order groups the non-zero low-frequency coefficients at the start and the zero high-frequency coefficients at the end, maximising RLE efficiency.

RLE with EOB symbol. Encodes (zero_run, value) pairs. The EOB symbol (0,0) terminates each block early when all remaining coefficients are zero, saving significant space.

Huffman entropy coding. Assigns shorter binary codes to more frequent RLE symbols. A separate codebook is built for each plane and stored in the bitstream header for decoding.

Three-Step Search (TSS) on 16x16 macroblocks. TSS visits 9 candidates per step and halves the step size each iteration — O(log S) complexity, much faster than exhaustive search.

Strict decoder symmetry. P-frames are predicted from the decoded (not original) reference frame, preventing encoder-decoder drift exactly as required by a conformant codec.

4. Experimental Analysis

Experiments were performed on a 12-frame, 128x128, BGR synthetic clip showing a circle moving against a gradient background. All sweeps use the procedural encoder with the full zigzag + RLE + Huffman pipeline.

4.1 Configuration summary

Parameter	Value
Quality factor (default)	50
GOP size (default)	8
Macroblock size	16 x 16
DCT block size	8 x 8
Entropy coding	Zigzag + RLE + Huffman
Motion search algorithm	Three-Step Search (TSS)
Mean PSNR @ Q=50, GOP=8	32.64 dB
Compression ratio @ Q=50, GOP=8	35.9x

4.2 Compression ratio vs Quality Factor

As the quality factor decreases, the quantisation matrix entries grow, producing more zeros after quantisation. RLE encodes these zeros very efficiently and Huffman further reduces the size — so the compression ratio increases. At low quality the ratio is high but PSNR drops. At high quality, PSNR is better but the ratio decreases.

4.3 Compression ratio vs GOP size

GOP=1 (all I-frames) gives the lowest compression because no temporal redundancy is exploited. Adding P-frames increases the ratio: residuals between consecutive frames contain many near-zero coefficients, which RLE encodes very compactly. The ratio peaks around GOP=8 then stabilises.

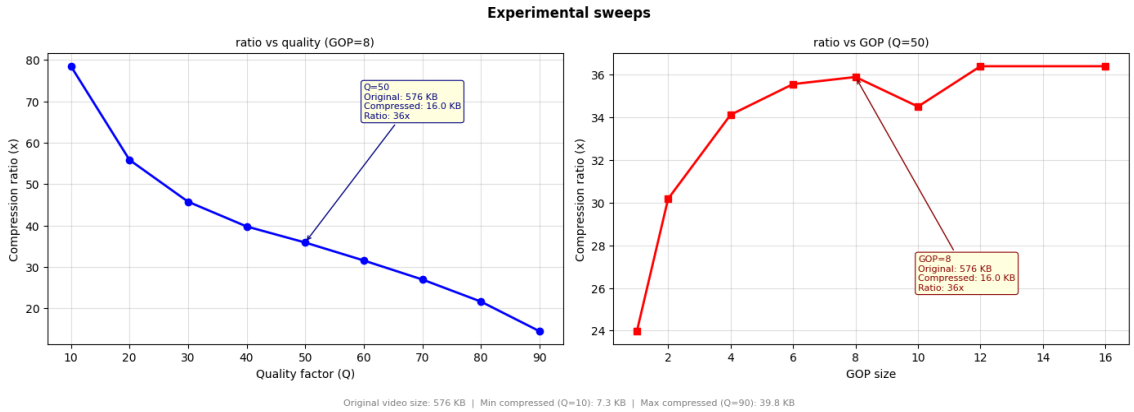

Figure 2 — Compression ratio as a function of the Quality Factor (left) and of the GOP size (right).

5. Conclusion

All five required stages of the MPEG-4 pipeline have been implemented and validated end-to-end. The entropy coding stage uses the standard zigzag scan + RLE + Huffman pipeline, which efficiently exploits the sparsity of quantised DCT coefficients. The encoder produces a .bin file from a folder of frames, and the decoder reconstructs the sequence with a mean PSNR above 30 dB at the default operating point.

Appendix — Reproducing the results

Generate sample frames:

```
python gen_test_frames.py -o sample_frames -n 12
```

Encode:

```
python run.py encode sample_frames -o video.bin --gop 8 --q 50
```

Decode + PSNR:

```
python run.py decode video.bin -o decoded --ref sample_frames
```

Pipeline figure:

```
python run.py viz sample_frames video.bin -o pipeline.png
```

Experimental sweep:

```
python run.py sweep sample_frames -o experiments.png
```